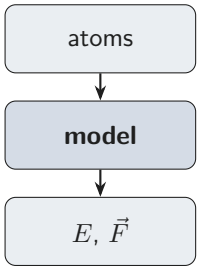
Training a Machine Learning Interatomic Potential

What we are building

A function from atoms to

- energy E (eV)
- force $\vec{F}_i$ per atom (eV/Å)
- with the right symmetries

Trained on DFT data; can reach DFT accuracy where hand-coded force fields plateau.



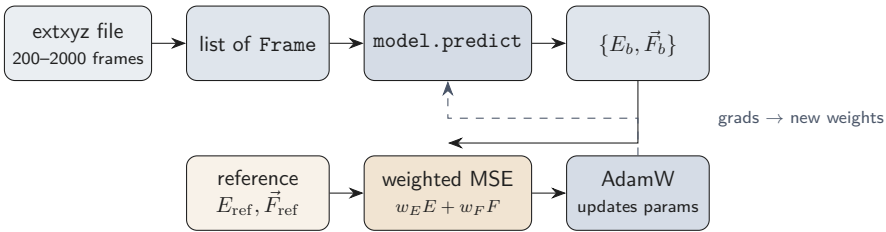
A "frame" is one configuration

```python
@dataclass
class Frame:
    positions: torch.Tensor        # (N, 3) Angstroms
    atomic_numbers: torch.Tensor   # (N,) long
    energy: torch.Tensor           # scalar, eV
    forces: torch.Tensor           # (N, 3), eV/A
    n_atoms: int
```

workshop1/frames.py:33

Our training set is 1600 frames of ethanol; validation is 400. Every frame has $N = 9$ atoms, E in eV, and DFT-computed forces.

The data path, end to end



`workshop1/train.py:166-180 (epoch loop)`

The training loop, in one screen

```
for epoch in range(epochs):
    for start in range(0, n, batch_size):
        batch = train_frames[order[start:start+batch_size]]
        opt.zero_grad()
        loss = w_E * MSE_E(batch) + w_F * MSE_F(batch)
        loss.backward()
        opt.step()
```

workshop1/train.py:166-180

Outer loop = **epoch**. Inner loop = **batch**.
Architecture, loss recipe, optimiser schedule all hang off this.

Epoch vs batch

Epoch

One full pass through the training set.

With 1600 train frames and batch size 200, that's $\lceil 1600/200 \rceil = 8$ gradient steps.

Reasonable target: $\sim$1000 epochs.

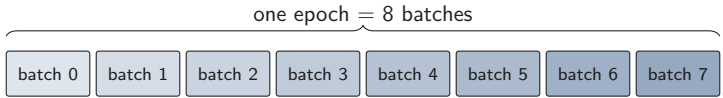
Batch

A subset of training data the optimiser sees in one step.

Larger batch

- less noisy gradient, smoother loss curve
- more memory, more GPU work per step
- fewer optimiser steps per epoch

For workshop1, full-batch is fine: 1600 frames
$\times$ 9 atoms is trivial.



Stochastic gradient descent

The loss is an average over the N training frames — so is its gradient:

$$L(\theta) = \frac{1}{N}\sum_{i=1}^{N} \ell_i(\theta) \qquad \nabla L(\theta) = \frac{1}{N}\sum_{i=1}^{N} \nabla \ell_i(\theta)$$

The exact gradient is N per-frame gradients, every step ($N = 1600$ here; millions for real datasets).

A **batch** of $B \ll N$ frames gives an unbiased estimate:

$$\nabla L(\theta) \;\approx\; \frac{1}{B}\sum_{i \in \text{batch}} \nabla \ell_i(\theta)$$

Cheaper per step, noisier direction, more steps — and the noise helps (regularises, escapes sharp minima). Full-batch ($B = N$) is exact; fine when N is small, as here.

The loss, exactly

```
def _batch_loss(model, batch, recipe):
    out = model.predict(batch)
    ref_E = stack([fr.energy for fr in batch])
    ref_F = stack([fr.forces for fr in batch])
    L_E = ((out["energies"] - ref_E) / n_atoms).pow(2).mean()
    L_F = (out["forces"]  - ref_F).pow(2).sum() / (3 * n_atoms * B)
    return recipe.w_E * L_E + recipe.w_F * L_F
```

workshop1/train.py:105-132

Two scalars per batch: L_E, L_F. We weight them and sum. Everything that follows — AdamW, the schedule, EMA — is just machinery for minimising this expression.

Why divide by N_{atoms}

Total energies grow with system size.
A 200-atom box has $\sim$200$\times$ the energy of a single molecule.

If we squared the raw error, the big box would dominate the loss and the small one would barely contribute.

Per-atom normalisation (E/N_{atoms}) puts every system on the same scale: a 9-atom ethanol and a 200-atom water cluster contribute equally, regardless of size.

`workshop1/train.py:120 ((out["energies"] - ref_E) / n_atoms)`

Forces are gradients

No special head for forces. We *differentiate* the energy.

$$\vec{F}_i = -\frac{\partial E}{\partial \vec{r}_i}$$

- `predict()` sets `positions.requires_grad_(True)`
- runs the energy forward pass
- calls `torch.autograd.grad(E, positions)`
- returns minus that gradient as the force

Energy conservation falls out for free.

`workshop1/models/pair_morse.py:153 (autograd.grad call)`

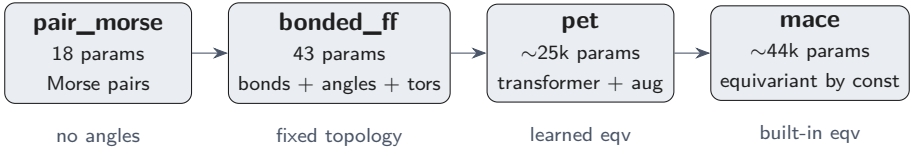
The recipe sweep

`train.py` runs the same model **five times**, varying only the loss weights:

recipe	w_E	w_F	teaches
E only	1	0	energies pin baseline, forces ignored
F only	0	1	forces drive shape; E off by a const
E:F = 1:1	1	1	balanced — both fit, both compromise
E:F = 1:100	1	100	MACE's canonical default
E:F = 100:1	100	1	energies dominate, forces drift

The loss you write down is the model you get.

Four architectures, one interface



Each implements one method:

```
def predict(self, frames):
    return {"energies": (B,) tensor,
            "forces":   list of (N_i, 3) tensors}
```

workshop1/models/__init__.py:1

Same loss recipe, different architectures

`compare_models.py` answers the orthogonal question.

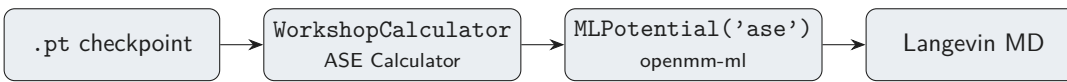
model	#params	RMSE_E meV/atom	RMSE_F meV/Å
pair_morse	18	high	high
bonded_ff	43	low	mid
pet	∼25k	low	mid
mace	∼44k	low	**best**

(run it for live numbers — they depend on epochs, precision, hardware)

- **pair_morse → bonded_ff:** angles enter; both metrics jump.
- **bonded_ff → pet:** far more params, marginal gain.
- **pet → mace:** similar budget, built-in equivariance, best forces.

From training to MD

Once you have a trained checkpoint:



Under the hood: `openmm.PythonForce` calls back into our calculator at every MD step. Same plumbing as ANI, NequIP, MACE-MP.

`workshop1/calculator.py`, `workshop1/md.py`

Is the PES actually right?

Two checks once you have a trained model:

Distributions (finite-T)
`workshop1.md_compare`

Run Langevin at 500 K, compare bond / angle / dihedral histograms against the training data.

A model with the wrong torsion barrier will redistribute mass across the dihedral histogram. Visible by eye.

Frequencies ($T = 0$)
`workshop1.vibrations`

Relax to the minimum, build the Hessian by finite differences, diagonalise.

For ethanol: OH stretch $\sim$3650, CH stretches 2900-3000, bends 1000-1500. A model that fits forces by accident gets these wrong.

The scripts

`workshop1.train`	loss recipe sweep, one architecture
`workshop1.compare_models`	architecture sweep, one recipe
`workshop1.data_efficiency`	val_F vs N_{train}
`workshop1.md`	short Langevin via openmm-ml
`workshop1.md_compare`	MD distributions vs training data
`workshop1.vibrations`	harmonic frequencies via Hessian

All share `frames.py`, the `models/` registry, and the same `predict(frames)` interface.

`workshop1/README.md`